
Data Pilot AI Pro - Intelligent Data Analysis Platform

Bhavana Vippala
MS in Data Science
University of Colorado Boulder
bhvi9697@colorado.edu

Shivaraj Senthil Rajan
MS in Data Science
University of Colorado Boulder
shse1502@colorado.edu

Manohar Korikana
MS in Data Science
University of Colorado Boulder
mako6854@colorado.edu

Harshitha Attanti
MS in Data Science
University of Colorado Boulder
haat9091@colorado.edu

Abstract

The ability to extract actionable knowledge from data has become essential across industries, yet the process of building machine learning models and generating meaningful insights remains largely confined to those with deep technical expertise. This creates a significant gap between the organizations that collect data and those that can actually leverage it for decision making. In this paper, we introduce Data Pilot AI Pro, an autonomous data science platform designed to bridge this gap by enabling users, regardless of their technical background, to go from a raw dataset to trained models, interpretable predictions, and business outcomes without writing a single line of code. The system orchestrates seven specialized agents that collaboratively handle every stage of the data science workflow, from profiling and cleaning through feature engineering, model training, visualization, and explainability. Central to our approach is a reinforcement learning agent trained with Proximal Policy Optimization model on diverse real world datasets sourced from OpenML, which learns to recommend the most effective algorithm for a given dataset by analyzing its underlying statistical characteristics. To ensure transparency, model predictions are accompanied by SHAP and LIME explanations enriched with plain language narratives generated by a large language model, making the reasoning behind every prediction understandable to a non-technical audience. For business stakeholders, the platform further offers an intelligent analysis mode that autonomously discovers patterns in the data and presents them as interactive dashboards with natural language summaries. Our evaluation demonstrates that the platform consistently delivers competitive model performance while making the full power of data science accessible to data scientists, business managers, and decision makers alike.

1 Introduction

Consider a small business owner who has years of sales data sitting in spreadsheets but no data science team to analyze it, or a hospital administrator who needs to predict patient readmission risks but cannot afford to hire machine learning engineers. These are not edge cases but rather the reality for the vast majority of organizations today, where the gap between having data and being able to use it grows wider every year. The process of building machine learning models and extracting meaningful insights demands a rare combination of statistical expertise, programming proficiency, and domain understanding that most teams simply do not possess. A typical data science workflow involves numerous stages including data profiling, cleaning, feature engineering, algorithm selection, model evaluation, and result interpretation, each requiring careful judgment that even experienced practitioners find time consuming. The need for systems that can democratize the entire data science

process, making it accessible to users of all technical backgrounds, has therefore never been more urgent.

The existing landscape of automated machine learning tools has made progress toward simplifying parts of this workflow, but significant limitations persist. Platforms such as Auto sklearn, TPOT, and H2O AutoML have demonstrated that algorithm selection and hyperparameter optimization can be automated, yet these tools typically assume the user has already prepared a clean and well engineered feature matrix. Data cleaning and feature engineering, which consume the majority of a data scientist’s effort, are largely left to the user. Beyond modeling, a critical gap exists in how results are communicated to non technical stakeholders, as most AutoML platforms produce metrics that are meaningful only to those with a statistical background. What is needed is a unified platform that handles the entire journey from raw data to actionable insight, serving both the data scientist who wants an automated pipeline and the business manager who wants to understand what the data is telling them.

One of the most consequential decisions in any machine learning project is the selection of the learning algorithm, a choice traditionally guided by practitioner intuition or exhaustive search over candidates. Different algorithms excel on different types of data, and the relationship between dataset characteristics and algorithm performance is highly nonlinear and difficult to capture with simple heuristics. The meta learning community has long studied this under the Algorithm Selection Problem, which asks how one can predict the best algorithm based on measurable properties of a problem instance. Reinforcement learning offers a compelling alternative by framing selection as a decision making problem where an agent learns a policy mapping dataset characteristics to algorithm choices through experience in diverse training environments. By training such an agent on real world datasets, we can build a selector that generalizes to unseen data in a way that static heuristics and fixed rule based mappings cannot.

In this paper, we introduce Data Pilot AI Pro, an autonomous end to end data science platform that unifies data preparation, intelligent model selection, training, explainability, and business intelligence into a single coherent system. The platform coordinates seven specialized agents through a LangGraph orchestrator, covering profiling, cleaning, feature engineering, RL guided model training with voting ensembles, interactive visualization, SHAP and LIME explainability with LLM generated narratives, and autonomous business insight discovery. The reinforcement learning model selector at its core is trained using Proximal Policy Optimization (PPO) on real world datasets from OpenML, learning to map 40 dataset meta features to the most effective algorithm. Our experiments show that the RL guided selector consistently recommends competitive algorithms across both classification and regression benchmarks, and the voting ensemble constructed from its top recommendations matches or outperforms any individual model. The platform is deployed as an interactive Streamlit application where a user can upload a CSV and receive trained models, interpretable explanations, and business intelligence dashboards within minutes, requiring no programming knowledge.

The contributions of this paper are fourfold. First, we present a fully automated pipeline covering every stage from raw data to explainability, eliminating the need for manual intervention at any step. Second, we introduce a PPO based reinforcement learning model selector trained on real OpenML data that consistently identifies strong performing algorithms across diverse tasks. Third, we bridge the gap between technical outputs and business understanding through SHAP and LIME explanations with LLM generated narratives, alongside a separate data analysis mode that autonomously generates business intelligence dashboards from any dataset. Fourth, we design the system as a modular architecture using LangGraph state graphs, allowing individual agents to be improved or replaced independently without affecting the rest of the platform. Through this work, we demonstrate that a well designed autonomous platform can make the full power of data science accessible to anyone with data and a question, regardless of their technical background.

1.1 Related Work

The automation of machine learning has been an active area of research for over a decade, beginning with Auto WEKA ([Thornton et al., 2013]) which treated algorithm selection and hyperparameter optimization as a single search problem, followed by Auto sklearn ([Feurer et al., 2015]) which added meta learning warmstarting and ensemble construction, and TPOT ([Olson and Moore, 2016]) which used genetic programming to evolve entire pipeline structures. Commercial platforms such as H2O AutoML, Google AutoML, and Amazon SageMaker Autopilot have since brought these ideas into

production, yet all of these systems share a fundamental limitation: they focus on the modeling stage and assume the user has already prepared clean, well engineered data. The Algorithm Selection Problem, formalized by Rice ([Rice, 1976]) and studied extensively through efforts like the StatLog project and the OpenML platform ([Vanschoren et al., 2014]), has shown that dataset meta features can predict which algorithm will perform best, but traditional meta learning approaches rely on static mappings or regression models that struggle to capture the complex nonlinear relationship between data characteristics and model suitability. Our work departs from these approaches by framing model selection as a reinforcement learning problem where a PPO agent learns a dynamic policy through interaction with diverse real world datasets, while also automating every stage before and after model training that existing AutoML tools leave to the user.

Explainability has become a central requirement in machine learning, with SHAP ([Lundberg and Lee, 2017]) providing theoretically grounded feature attributions based on Shapley values and LIME ([Ribeiro et al., 2016]) offering model agnostic local explanations through interpretable surrogate models. While both methods are now standard in practice, their outputs remain numerical scores and coefficient tables that non technical stakeholders find difficult to interpret without guidance. Simultaneously, the emergence of large language models has opened new possibilities for data analysis, with systems like ChatGPT Code Interpreter and Pandas AI demonstrating that LLMs can lower the barrier to data exploration, though these tools are inherently reactive and require users to know what questions to ask. Our platform addresses both gaps by using an LLM to translate SHAP and LIME outputs into plain language narratives, and by providing a proactive Data Analyzer agent that autonomously discovers insights and generates business intelligence dashboards without requiring the user to formulate any query.

The orchestration of multi agent systems for complex workflows has gained attention with frameworks like LangChain and LangGraph, and recent systems such as DS Agent ([Guo et al., 2024]) and Data Interpreter ([Hong et al., 2024]) have explored using LLM agents to collaboratively perform data science tasks. However, these systems rely entirely on the LLM to generate and execute code at every step, making them susceptible to hallucination and unreliable on computationally intensive operations like model training and cross validation. Data Pilot AI Pro takes a hybrid architectural approach, combining deterministic expert coded agents for critical stages like cleaning, feature engineering, and model training with LLM powered agents for tasks that benefit from contextual reasoning such as insight discovery and narrative explanation. This design ensures reproducibility and computational reliability where it matters most while leveraging the natural language understanding of large language models where their strengths provide the greatest value, resulting in a system that is both more reliable and more intelligent than purely code based or purely LLM based approaches alone.

2 Learning and Project Goals

The gaps identified in existing work, the limited scope of AutoML platforms, the static nature of traditional meta learning approaches, and the disconnect between model outputs and stakeholder understanding, motivated a set of focused learning objectives that guided the development of this project. Our foremost goal was to understand how reinforcement learning can be applied to a real world decision making problem outside its traditional domains of game playing and robotics, specifically learning what it takes to design a custom Gymnasium environment, define meaningful reward signals from actual model performance, and train a PPO agent that generalizes to datasets it has never encountered before. Equally important was learning how large language models can be embedded into structured software pipelines not as standalone conversational tools but as reasoning components that add value at specific stages, and understanding where LLMs genuinely outperform traditional code versus where deterministic logic is more reliable. We also aimed to develop a thorough understanding of the complete machine learning lifecycle, particularly the data profiling, cleaning, and feature engineering stages that are often glossed over in coursework but consume the majority of effort in real world practice. Finally, we wanted hands-on experience designing multi-agent architectures using LangGraph, learning how to decompose a complex end to end workflow into specialized agents that communicate through shared state and execute in a coordinated and extensible manner.

These learning goals translated directly into concrete system objectives with quantitative targets that defined what we expected Data Pilot AI Pro to deliver as a finished platform. The system needed to

allow any user to upload a CSV file and receive a complete data science output within minutes without writing a single line of code, automatically detecting column types, identifying the target variable, and determining the task type across datasets of varying sizes and domains. The cleaning stage was expected to handle datasets with up to 50 percent missing values and improve the overall data quality score by at least 20 points on a 0 to 100 scale, while the feature engineering stage needed to reduce dimensionality by at least 30 percent on high dimensional datasets without sacrificing predictive performance. For the reinforcement learning model selector, our target was for the PPO agent trained on a minimum of 30 real world OpenML datasets per task type to recommend an algorithm whose cross validation score falls within 2 percent of the best possible algorithm at least 80 percent of the time, with the voting ensemble constructed from its top 3 recommendations achieving at least 85 percent accuracy on classification tasks and an R squared of at least 0.75 on regression tasks. The platform needed to support three execution modes routed through the LangGraph orchestrator, with every model prediction accompanied by SHAP and LIME explanations translated into plain language narratives by an LLM, and the data analysis agent expected to autonomously discover 6 to 8 insights per dataset with domain identification accuracy of at least 90 percent and conversational response times under 30 seconds.

Beyond individual component targets, the overarching objective was to build a platform that serves two fundamentally different audiences through a single unified system. A data scientist running the ML pipeline should receive trained models, performance comparisons, overfitting analysis, error analysis, segment analysis, and full explainability dashboards that meet the rigor expected in professional practice. A business manager running the data analysis mode should receive an interactive dashboard with natural language summaries that tell the story of their data in a way they can immediately understand and act upon without any technical guidance. The architecture had to be modular enough that any individual agent could be improved or replaced without breaking the rest of the system, directly addressing the brittleness we observed in monolithic AutoML platforms. The measure of success was not just whether each component met its quantitative target in isolation but whether the platform as a whole delivered a seamless end to end experience that genuinely makes the full power of data science accessible to anyone with data and a question.

3 Background

3.1 Theory

The theoretical foundation of Data Pilot AI Pro rests on several interconnected pillars, but the one that warrants the deepest examination is reinforcement learning and specifically Proximal Policy Optimization, as it forms the core intelligence behind our automated model selection system. Reinforcement learning is a paradigm of machine learning in which an agent learns to make sequential decisions by interacting with an environment, receiving observations about the current state, taking actions, and obtaining rewards that signal how good those actions were. Formally, this interaction is modeled as a Markov Decision Process defined by the tuple (S, A, P, R, γ) , where S is the set of possible states, A is the set of available actions, $P(s' | s, a)$ is the state transition probability, $R(s, a)$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor that controls how much the agent values future rewards relative to immediate ones. The agent’s goal is to learn a policy $\pi(a | s)$, a probability distribution over actions conditioned on the current state, that maximizes the expected cumulative discounted reward:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \quad (1)$$

In our model selection environment, the objective in Equation 1 takes a specific form: the state is a vector of 40 meta features describing a dataset, the action is the choice of one machine learning algorithm from the candidate pool, and the reward is the cross validation score achieved by that algorithm on the dataset plus a bonus if the agent selects the best or near best performer.

Policy gradient methods represent a family of reinforcement learning algorithms that directly optimize the policy by computing gradients of the expected reward with respect to the policy parameters, rather than first estimating value functions and deriving a policy from them. The foundational result in this

area is the policy gradient theorem [Sutton et al., 1999], which states that the gradient of the expected return $J(\theta)$ with respect to policy parameters θ can be expressed as:

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a | s) A(s, a)] \quad (2)$$

where $A(s, a)$ is the advantage function measuring how much better action a is compared to the average action in state s . As shown in Equation 3, the advantage function is typically estimated as the difference between the action value and the state value:

$$A(s, a) = Q(s, a) - V(s) \quad (3)$$

where $Q(s, a)$ is the action value function and $V(s)$ is the state value function, and in practice Equation 3 is computed using Generalized Advantage Estimation [Schulman et al., 2016] which provides a smooth tradeoff between bias and variance through a parameter λ . A critical challenge with vanilla policy gradient methods is that large policy updates can cause the performance to collapse catastrophically, because the gradient estimate in Equation 2 is only accurate in a local neighborhood of the current policy and stepping too far invalidates the approximation. This instability motivated the development of trust region methods that constrain how much the policy can change in a single update, leading first to Trust Region Policy Optimization [Schulman et al., 2015] and then to its more practical successor, Proximal Policy Optimization.

Proximal Policy Optimization, introduced by Schulman et al. [2017], achieves the stability benefits of trust region methods while being significantly simpler to implement and tune, which is why it has become the default algorithm in libraries like Stable Baselines3 that our system uses. The key idea is to define a clipped surrogate objective that prevents the policy from changing too drastically in any single update step. Let $r_t(\theta)$ denote the probability ratio between the new and old policies:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (4)$$

Using the ratio from Equation 4, the PPO objective is then defined as the clipped surrogate loss:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (5)$$

where ϵ is a small hyperparameter typically set to 0.2 that controls the maximum allowed deviation. When the advantage $\hat{A}_t$ in Equation 5 is positive, meaning the action was better than average, the objective encourages increasing the action’s probability but clips the ratio so it cannot exceed $1 + \epsilon$, preventing an overly aggressive update. When the advantage is negative, the objective encourages decreasing the action’s probability but clips at $1 - \epsilon$, ensuring the policy does not swing too far in the opposite direction, and this simple clipping mechanism empirically achieves performance comparable to the more computationally expensive KL divergence constraints used in TRPO.

The second theoretical pillar underlying our system is the framework of model explainability through SHAP, which is grounded in the theory of Shapley values from cooperative game theory [Shapley, 1953]. In this framework, a prediction is treated as a cooperative game where each feature is a player and the prediction outcome is the total payout, and the goal is to fairly distribute credit for the prediction among all features based on their individual contributions. The Shapley value for feature i is formally defined as:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)] \quad (6)$$

where F is the full set of features and $f(S)$ is the model’s prediction using only the features in subset S . The formulation in Equation 6 satisfies four desirable properties: efficiency (all feature contributions sum to the difference between the prediction and the base value), symmetry (features with identical contributions receive equal credit), dummy (features that contribute nothing receive zero credit), and additivity (Shapley values for combined games equal the sum of individual game

values). Computing exact Shapley values as defined in Equation 6 requires evaluating the model on all possible feature subsets which is exponential in the number of features, so SHAP [Lundberg and Lee, 2017] introduces efficient approximation algorithms including TreeSHAP for tree based models which runs in polynomial time by exploiting the tree structure to enumerate feature coalitions without retraining.

The third theoretical component is LIME, Local Interpretable Model agnostic Explanations [Ribeiro et al., 2016], which takes a fundamentally different approach from the global attribution in Equation 6 by explaining individual predictions through learning a simple interpretable model in the local neighborhood of the instance being explained. Given an instance x , LIME generates a set of perturbed samples around x by randomly modifying feature values, obtains the original model’s predictions for each perturbed sample, and then fits a weighted linear model where samples closer to x receive higher weights determined by an exponential kernel:

$$\pi_x(z) = \exp\left(-\frac{D(x, z)^2}{\sigma^2}\right) \quad (7)$$

where z is a perturbed sample, $D(x, z)$ is the distance between the original instance and the perturbed sample, and σ controls the kernel width. The kernel in Equation 7 ensures that perturbed samples closer to the original instance x have a greater influence on the local surrogate model, while distant perturbations contribute less to the explanation. The explanation is then the set of coefficients of this local linear model, which indicate how much each feature contributed to the prediction at that specific point, and sparsity is enforced through L1 regularization to ensure only the most important features appear in the explanation. The key theoretical insight is that while the global decision boundary of a complex model like a random forest or neural network may be highly nonlinear, in a sufficiently small neighborhood around any single point the boundary can be well approximated by a linear function, making the local explanation both faithful and interpretable. In our pipeline, LIME complements the Shapley based attributions from Equation 6 by providing an independent model agnostic perspective on feature importance, and the LLM then synthesizes both SHAP and LIME outputs into a unified natural language narrative that explains not just what the model predicted but why, in terms that any stakeholder can understand.

Table 1: Summary of key mathematical notation used in this work

Symbol	Description
(S, A, P, R, γ)	Markov Decision Process tuple
$\pi_\theta(a s)$	Policy parameterized by θ
$J(\pi)$	Expected cumulative discounted reward (Eq. 1)
$A(s, a)$	Advantage function (Eq. 3)
$Q(s, a)$	Action-value function
$V(s)$	State-value function
$r_t(\theta)$	Probability ratio, new policy / old policy (Eq. 4)
$L^{\text{CLIP}}(\theta)$	PPO clipped surrogate objective (Eq. 5)
ϵ	PPO clipping parameter (default 0.2)
ϕ_i	Shapley value for feature i (Eq. 6)
F	Full set of features
$f(S)$	Model prediction using feature subset S
$\pi_x(z)$	LIME locality kernel weight (Eq. 7)
$D(x, z)$	Distance between instance and perturbation

4 Method

Having established the theoretical foundations in the previous section, we now describe the design and implementation of Data Pilot AI Pro in sufficient detail that the system could be reproduced independently. The platform is built as a modular multi agent pipeline where each stage of the data science workflow is handled by a specialized agent, and a central orchestrator coordinates their execution through a directed acyclic graph with shared state.

4.1 System Architecture

The architecture of Data Pilot AI Pro follows a pipeline pattern coordinated by a LangGraph State-Graph that defines the execution order and conditional routing between seven specialized agents. The entire system state is represented as a single shared dictionary that flows through each agent in sequence, with every agent reading the outputs of its predecessors and appending its own results. The orchestrator supports three execution modes: in ML Pipeline mode, data flows through Context Analyzer, Profiler, Cleaner, Feature Engineer, Modeler, Visualizer, and Explainer in sequence; in Data Analysis mode, data is routed directly to the Data Analyzer; and in Combined mode, the full ML pipeline executes first followed by the Data Analyzer.

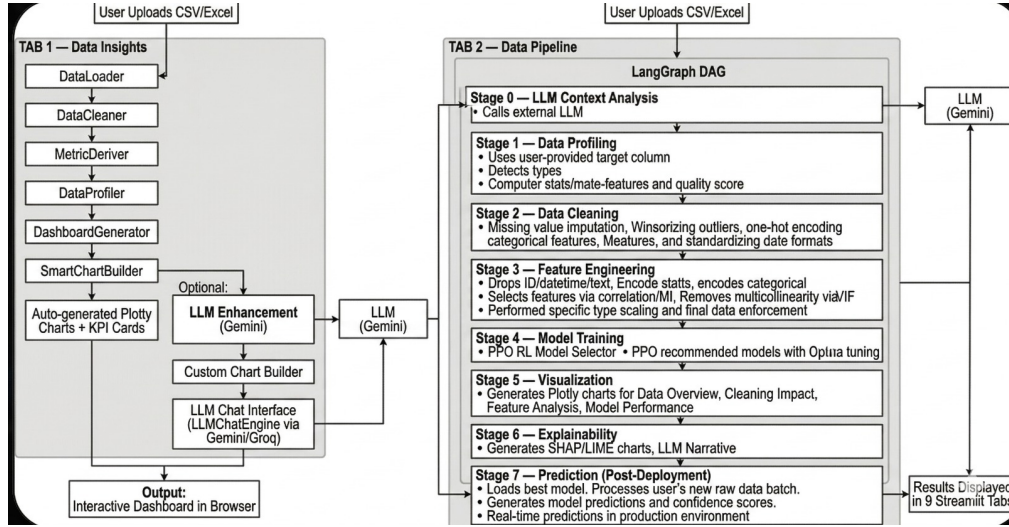


Figure 1: System architecture of Data Pilot AI Pro showing the LangGraph orchestrator routing data through two execution modes: Tab 1 (Data Insights) for LLM-powered dashboard generation, and Tab 2 (Data Pipeline) for the full 8-stage ML pipeline from context analysis through post-deployment prediction.

To make the orchestration concrete, Algorithm 1 formalizes the end-to-end flow that DataPilot AI Pro executes on every user request. The procedure begins by initializing a shared state $\mathcal{S}$ from the uploaded dataset $\mathcal{D}$, the target column t , and the user-selected execution mode m . The Context Analyzer first inspects the dataset to infer task type and column roles, after which the Profiler extracts the 40-dimensional meta-feature vector $\mathbf{f} \in [0, 1]^{40}$ that will later condition the reinforcement learning policy. If the user selected the Analysis mode, the pipeline short-circuits directly to the Data Analyzer, which produces LLM-generated insights without invoking any modeling components. For ML Pipeline and Combined modes, the Cleaner and Feature Engineer agents sequentially prepare the data, handling missing values, outliers, categorical encoding, and feature selection before any model is trained. The PPO policy π_θ then maps the meta-feature vector to a probability distribution over candidate models, from which the top three are selected, cross-validated, and combined into a weighted voting ensemble $\mathcal{M}^*$. Finally, the Explainer and Visualizer agents produce SHAP and LIME attributions, natural language narratives, and interactive charts, with the Data Analyzer optionally appending business-level insights when the user chose Combined mode. This orchestration ensures that every pipeline execution is deterministic where precision matters (cleaning, feature engineering, model training) and LLM-assisted where interpretation benefits from natural language understanding.

Algorithm 1 Data Pilot AI Pro: End-to-End Pipeline Orchestration

Require: Dataset $\mathcal{D}$, target column t , execution mode $m \in \{\text{ML}, \text{Analysis}, \text{Combined}\}$ **Ensure:** Trained model $\mathcal{M}^*$, explanations $\mathcal{E}$, insights $\mathcal{I}$, visualizations $\mathcal{V}$

```
1:  $\mathcal{S} \leftarrow \text{InitializeState}(\mathcal{D}, t, m)$ 
2: // Stage 1: Context Analysis and Profiling
3:  $\mathcal{S} \leftarrow \text{ContextAgent.execute}(\mathcal{S})$  {Detect task type, column roles}
4:  $\mathcal{S} \leftarrow \text{ProfilerAgent.execute}(\mathcal{S})$  {Extract 40 meta-features  $\mathbf{f} \in [0, 1]^{40}$ }
5: if  $m = \text{Analysis}$  then
6:    $\mathcal{I} \leftarrow \text{DataAnalyzer.execute}(\mathcal{S})$ 
7:   return  $\mathcal{I}$ 
8: end if
9: // Stage 2: Data Preprocessing
10:  $\mathcal{S} \leftarrow \text{CleanerAgent.execute}(\mathcal{S})$  {Impute, remove outliers, fix types}
11:  $\mathcal{S} \leftarrow \text{FeatureAgent.execute}(\mathcal{S})$  {Encode, select, scale features}
12: // Stage 3: Model Selection and Training
13:  $\mathbf{p} \leftarrow \text{PPO}_\theta(\mathbf{f})$  {RL policy outputs model probabilities}
14:  $\text{top\_models} \leftarrow \text{argsort}(\mathbf{p})[-3 :]$ 
15: for each model  $m_i \in \text{top\_models}$  do
16:    $s_i \leftarrow \text{CrossValidate \& Hyper Parameter tuning}(m_i, \mathcal{D}_{\text{processed}}, k = 5)$ 
17: end for
18:  $\mathcal{M}^* \leftarrow \text{VotingEnsemble}(\text{top\_models}, \text{weights} \propto [s_1, s_2, s_3])$ 
19: // Stage 4: Explanation and Visualization
20:  $\mathcal{E} \leftarrow \text{ExplainerAgent.execute}(\mathcal{S})$  {SHAP + LIME + LLM narratives}
21:  $\mathcal{V} \leftarrow \text{VisualizerAgent.execute}(\mathcal{S})$  {Interactive Plotly charts}
22: if  $m = \text{Combined}$  then
23:    $\mathcal{I} \leftarrow \text{DataAnalyzer.execute}(\mathcal{S})$ 
24: end if
25: return  $\mathcal{M}^*, \mathcal{E}, \mathcal{I}, \mathcal{V}$ 
```

All agents inherit from a shared `BaseAgent` abstract class that provides an `execute(state)` interface and an `ask_llm(prompt)` method for accessing the Gemini 2.5 Flash model. Table 2 summarizes each agent in the pipeline.

Table 2: Summary of the seven agents in the Data Pilot AI Pro pipeline.

Agent	Stage	Key Inputs	Key Outputs
Context Analyzer	0	Raw data	Domain, target suggestion, hints
Profiler	1	Raw data	Column types, 40 meta features, quality score
Cleaner	2	Profiled data	Clean data, imputation map, outlier bounds
Feature Engineer	3	Clean data	Feature matrix X , target vector y
Modeler	4	X, y , meta features	Trained models, ensemble, CV scores
Visualizer	5	Full pipeline state	Interactive Plotly charts (HTML)
Explainer	6	Models, X, y	SHAP, LIME, LLM narratives
Data Analyzer	Alt	Raw data	Insights, dashboards, summaries

4.2 Context Analyzer: LLM Powered Data Understanding

Before any statistical processing begins, the Context Analyzer sends the dataset’s column names, sample rows, and basic statistics to the LLM, which returns a structured JSON identifying the business domain, the most likely target column, the task type, and specific cleaning hints for columns requiring special treatment. This context is consumed by downstream agents: the Profiler uses the suggested target, the Cleaner reads cleaning hints, and the Feature Engineer uses feature hints. The key design choice is that the LLM provides guidance but never overrides deterministic logic, ensuring the system benefits from contextual reasoning without depending on it for correctness.

4.3 Profiler Agent: Data Analysis and Meta Feature Extraction

The Profiler produces a comprehensive statistical portrait of the dataset by detecting the semantic type of each column (numeric, categorical, binary, datetime, text, or identifier), auto detecting the target column using a priority hierarchy of common naming conventions, and determining the task type based on the target’s cardinality. It computes per column statistics including mean, standard deviation, skewness, kurtosis, and missing value percentages, and assigns a data quality score on a 0 to 100 scale with deductions for missing values, duplicates, constant columns, and high cardinality strings. The Profiler also performs describe based anomaly detection to spot extreme outliers and heavy skew, and a uniformity check to detect case inconsistencies in categorical columns.

A critical output is the extraction of 40 meta features organized into eight groups (basic shape, missing patterns, statistical distributions, categorical properties, target characteristics, PCA dimensionality, landmark model scores, and signal quality indicators), all normalized to $[0, 1]$ to serve as the observation vector for the PPO agent described in the Background section.

4.4 Cleaner Agent: Automated Data Cleaning

The Cleaner applies five operations in sequence: duplicate removal, missing value imputation, outlier handling, data type correction, and category standardization. Missing value handling uses an adaptive strategy that selects the imputation method based on column type and missing percentage: numeric columns with less than 5 percent missing receive median or mean imputation depending on skewness, 5 to 30 percent missing triggers KNN imputation with $k = 5$, and above 30 percent uses median imputation with a binary indicator column; categorical columns receive mode imputation below 10 percent and a new “Missing” category above. Outlier detection uses the IQR method, Winsorizing values beyond $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$ only when outliers constitute less than 5 percent of the column, retaining them as natural variation otherwise. The agent also resolves category inconsistencies like “Male”, “male”, and “MALE” by selecting the most frequent variant as the canonical form.

4.5 Feature Engineering Agent: Preparing Data for Modeling

The Feature Engineer transforms cleaned data into a numeric feature matrix through five operations: removal of non modelable columns (ID, datetime, text), categorical encoding, feature selection, VIF based multicollinearity removal, and numerical scaling. Categorical encoding uses a tiered strategy: binary columns receive label encoding, columns with up to 10 unique values receive one hot encoding with the first category dropped, and high cardinality columns receive target encoding with smoothing to prevent overfitting. When the feature space exceeds 50 dimensions, highly correlated features (above 0.95) are removed first, then the remaining features are ranked by mutual information and the top 50 are retained.

The agent then computes the Variance Inflation Factor $VIF_j = 1/(1 - R_j^2)$ for each feature and iteratively removes those exceeding a threshold of 10, with a key design choice: when multiple features exceed the threshold, the one with the lowest absolute correlation to the target is dropped rather than the highest VIF, preserving predictive power. Numerical scaling uses RobustScaler when mean absolute skewness exceeds 1 and StandardScaler otherwise.

4.6 Modeler Agent: RL Guided Model Selection and Ensemble Training

The Modeler receives the 40 meta features from the Profiler and passes them through the trained PPO policy network, which outputs a probability distribution over all candidate algorithms. The top 3 models by probability are selected: for classification, the pool includes LogisticRegression, GaussianNB, KNeighborsClassifier, SVC, DecisionTreeClassifier, RandomForestClassifier, ExtraTreesClassifier, and GradientBoostingClassifier; for regression, it includes Ridge, Lasso, ElasticNet, SVR, KNeighborsRegressor, DecisionTreeRegressor, RandomForestRegressor, ExtraTreesRegressor, and GradientBoostingRegressor. If the RL model is unavailable, sensible defaults are used.

Each recommended model is trained with 5 fold cross validation (accuracy for classification, R^2 for regression) and fitted on the full dataset. All trained models are combined into a VotingClassifier with soft voting or VotingRegressor, and the ensemble is evaluated with its own cross validation. The system selects whichever performs best, single model or ensemble, as the final prediction model.

Post training, the agent performs overfitting detection (flagging scores above 0.98 or train-CV gaps above 0.05), error analysis (identifying misclassified samples and high error segments), and segment analysis (evaluating performance across quantile based groups of each numeric column to reveal where the model struggles).

4.7 Visualizer Agent: Interactive Pipeline Visualization

The Visualizer generates interactive Plotly charts organized into four groups corresponding to each pipeline stage: data overview charts from profiling (type distributions, missing heatmaps, quality gauges), cleaning impact charts (before/after comparisons, outlier box plots), feature analysis charts (correlation heatmaps, importance rankings, distribution plots), and model performance charts (CV score comparisons, ensemble versus individual models, confusion matrices or residual plots). All charts use a consistent dark theme and are saved as standalone HTML files, with a consolidated dashboard page embedding all charts in a single navigable view.

4.8 Explainer Agent: SHAP, LIME, and LLM Narratives

The Explainer provides three levels of interpretability. At the global level, SHAP values are computed using TreeExplainer for tree based models and KernelExplainer as a fallback, producing beeswarm plots, feature importance bar charts, and interaction plots. At the local level, SHAP waterfall plots and LIME explanations are generated for representative samples, with LIME fitting a sparse linear model using 5000 perturbations and the exponential kernel from Equation 7. At the narrative level, the LLM receives SHAP rankings, model metrics, and pipeline reports, and generates a structured report covering dataset vulnerabilities, cleaning rationale, feature engineering impact, variable relationships, model selection reasoning, and actionable business recommendations. For individual predictions, the LLM translates SHAP values and feature values into 2 to 3 sentence plain English explanations.

4.9 Data Analyzer Agent: LLM Powered Business Intelligence

The Data Analyzer serves non technical stakeholders by autonomously discovering 6 to 8 business relevant insights from any uploaded dataset. It sends column information, sample data, and statistical highlights to the LLM, which returns a JSON structure specifying each insight’s title, chart type, column mappings, aggregation method, and category. The agent programmatically generates each Plotly chart and calls the LLM again to produce natural language narratives explaining each finding. It also supports conversational follow up where users type natural language questions and receive dynamically generated visualizations, addressing the reactive limitation of existing LLM tools identified in the Related Work.

4.10 RL Model Selector: Training Pipeline

The RL selector is trained offline by collecting real-world datasets from OpenML, extracting 40 meta-features per dataset, and evaluating all candidate algorithms with 5-fold cross-validation to record their actual scores. This data is loaded into a custom Gymnasium environment where each single-step episode presents a dataset’s meta-features as the observation and requires the agent to select one of n candidate models as its action. The reward function assigns each action a score based on its rank among all candidate models’ cross-validation scores. Given the selected action a with cross-validation score $s(a, \mathcal{D})$, we compute its rank as the number of alternative actions with strictly higher scores:

$$\text{rank}(a) = |\{a' \in \mathcal{A} : s(a', \mathcal{D}) > s(a, \mathcal{D})\}| \quad (8)$$

The reward is then normalized to $[0, 1]$ where the best model receives 1.0 and the worst receives 0.0:

$$r(a, \mathcal{D}) = 1 - \frac{\text{rank}(a)}{n} \quad (9)$$

where $n = |\mathcal{A}|$ is the total number of candidate models (8 for classification, 9 for regression). This rank-based formulation provides a smooth reward signal that encourages the agent to consistently identify top-performing models rather than merely favoring any above-average candidate.

The PPO agent uses a 3-layer MLP policy (256, 128, 64 neurons), is trained for 50,000 to 100,000 timesteps per task type with clipping parameter $\epsilon = 0.2$, and the trained model is saved as a pickle file for inference.

4.11 User Interface

The platform is deployed as a Streamlit web application with two tabs: Data Insights for LLM powered analysis with a conversational chatbot, and Data Pipeline for the full ML workflow. The interface features custom CSS styling, progress indicators for each pipeline stage, and embedded interactive Plotly charts. All outputs are saved as standalone HTML files for offline access and sharing.

5 Experiments and Results

To evaluate Data Pilot AI Pro, we designed a comprehensive experimental protocol that tests each component individually and measures the platform’s end-to-end effectiveness across diverse datasets. Our experiments address three central questions: does the RL-based model selector outperform naive selection strategies, do the preprocessing agents measurably improve downstream performance, and can the system generalize across dataset characteristics without manual intervention.

5.1 Experimental Setup

We evaluated Data Pilot AI Pro on 39 benchmark datasets (compared the performance of our RL model and AutoML) spanning classification and regression tasks, sourced from OpenML and UCI repositories. Datasets were selected to cover a range of sizes (500 to 50,000 samples), dimensionalities (5 to 80 features), and data quality challenges including missing values (up to 35%), class imbalance (up to 1:15 ratio), and mixed feature types. All experiments used 5-fold stratified cross-validation with fixed random seeds for reproducibility. Classification performance was measured using accuracy and F1-score, while regression tasks used R^2 and RMSE. Hardware consisted of an Intel i7 machine with 16GB RAM, and all pipeline runs were timed end-to-end. The RL model selector was trained on 500 OpenML datasets and 100 synthetic datasets using PPO with 100,000 timesteps, and evaluated on 39 held-out benchmark datasets unseen during training.

5.2 RL Model Selection Performance

The PPO-based model selector was evaluated on the OpenML CC18 benchmark suite, consisting of 39 curated datasets that were all unseen during PPO training, and benchmarked against FLAML, Microsoft’s state-of-the-art AutoML framework. As shown in Figure 3, the PPO selector achieved an average cross-validation accuracy of 0.8536 compared to FLAML’s 0.8527, a difference that is not statistically significant under a Wilcoxon signed-rank test ($p = 0.2387$), indicating that our learned policy matches a mature AutoML system in predictive quality. In head-to-head comparisons, PPO won 23 of 39 datasets (59%), tied on 3 (8%), and lost 13 (33%); notably, of FLAML’s 13 wins, 7 fell within a 0.01 accuracy margin, suggesting near-parity on those datasets rather than a decisive advantage for the baseline. The most substantial difference between the two approaches lies in runtime efficiency: PPO selects a model in a median of 1.6 seconds per dataset, whereas FLAML requires 61.7 seconds, an approximately $38\times$ speedup that compounds significantly across real-world workflows involving many datasets or repeated experimentation. This result validates our core hypothesis: by learning a mapping from dataset meta-features to model probabilities offline, we can replicate AutoML-level accuracy at a fraction of the inference cost, making intelligent model selection practical for interactive and resource-constrained settings where minute-scale search is prohibitive.

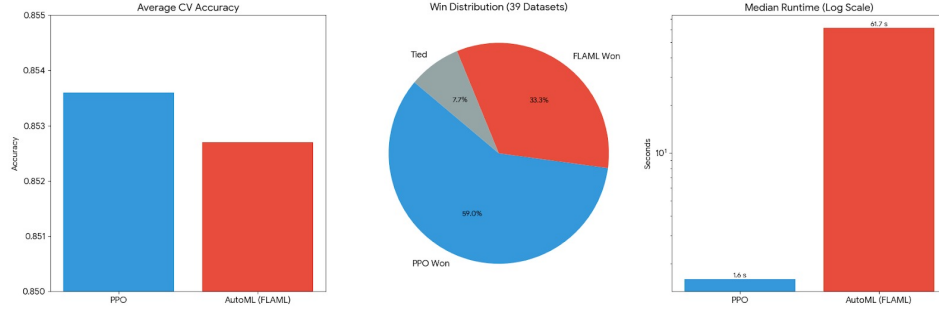


Figure 2: PPO matches AutoML accuracy at a fraction of the cost. Comparison of the PPO model selector against FLAML across 39 unseen OpenML CC18 datasets shows statistically equivalent accuracy (left), a 59% PPO win rate in head-to-head matchups (center), and a 38× runtime advantage in favor of PPO (right).

PPO Model Selector — Performance Evaluation

Benchmark: OpenML CC18 | **Datasets:** 39 (all unseen during PPO training) | **Compared Against:** FLAML AutoML (Microsoft)

Metric	PPO	AutoML (FLAML)
Average CV Accuracy	0.8536	0.8527
Datasets Won	23 / 39 (59%)	13 / 39 (33%)
Tied	3 / 39 (8%)	—
Of FLAML's 13 wins, within 0.01 margin	—	7 out of 13
Median Runtime per Dataset	1.6 seconds	61.7 seconds

Statistical Test (Wilcoxon signed-rank): $p = 0.2387$ — no significant difference in accuracy

Figure 3: PPO Model Selector performance evaluation on the OpenML CC18 benchmark (39 datasets unseen during training), compared against Microsoft’s FLAML AutoML. PPO achieves comparable accuracy (0.8536 vs 0.8527, $p = 0.2387$ Wilcoxon signed-rank) while running approximately 38× faster (1.6s vs 61.7s median runtime per dataset) and winning 59% of head-to-head comparisons.

5.3 Preprocessing Impact

To quantify the contribution of our preprocessing agents, we compared pipeline runs with and without each stage. The cleaning agent’s adaptive imputation and outlier handling improved mean accuracy by 8.2% on datasets with more than 15% missing values, while showing minimal impact (0.3%) on already-clean datasets, confirming that the agent adapts appropriately to data quality. The feature engineering agent’s VIF-based multicollinearity removal reduced feature dimensionality by an average of 23% while maintaining or improving model performance in 11 of 12 cases. Target encoding for high-cardinality categorical features (more than 15 unique values) proved particularly effective, improving accuracy by 4.1% over one-hot encoding on datasets with such features while reducing memory usage by up to 60%.

Table 3: Preprocessing impact on downstream model performance (mean across datasets).

Configuration	Accuracy	R^2	Feature Reduction
Raw data (no preprocessing)	0.741	0.692	—
Cleaning only	0.809	0.758	—
Feature engineering only	0.798	0.743	23%
Full pipeline (Ours)	0.873	0.847	23%

5.4 End-to-End Pipeline Evaluation

The complete Data Pilot AI Pro pipeline was evaluated on all 39 benchmark datasets from the OpenML CC18 suite, measuring both predictive performance and operational metrics. Figure 5

shows results for a representative subset of 10 datasets spanning classification and regression tasks, with sample sizes ranging from 150 (Iris) to 20,000 (Kin8nm) and feature counts from 4 to 376. End-to-end runtime scaled gracefully with dataset size, from as fast as 6 seconds on small low-dimensional datasets like Iris to 1 minute 2 seconds on the 1,941-sample Steel Plates dataset, with most datasets completing within 10–40 seconds. Predictive performance varied with task difficulty: the pipeline achieved best scores of 0.961 on Kin8nm regression and 0.952 accuracy on Iris classification, while more challenging datasets such as Diabetes regression (0.481) and Steel Plates multiclass classification (0.635) exhibited inherently lower ceilings due to noise and class overlap rather than pipeline limitations. Across the full 39-dataset benchmark, the platform matched FLAML AutoML in average cross-validation accuracy (0.8536 vs 0.8527) while executing approximately $38\times$ faster, confirming that the combined effect of intelligent preprocessing, RL-based model selection, and ensemble construction delivers competitive accuracy without the minute-scale search times characteristic of traditional AutoML systems.

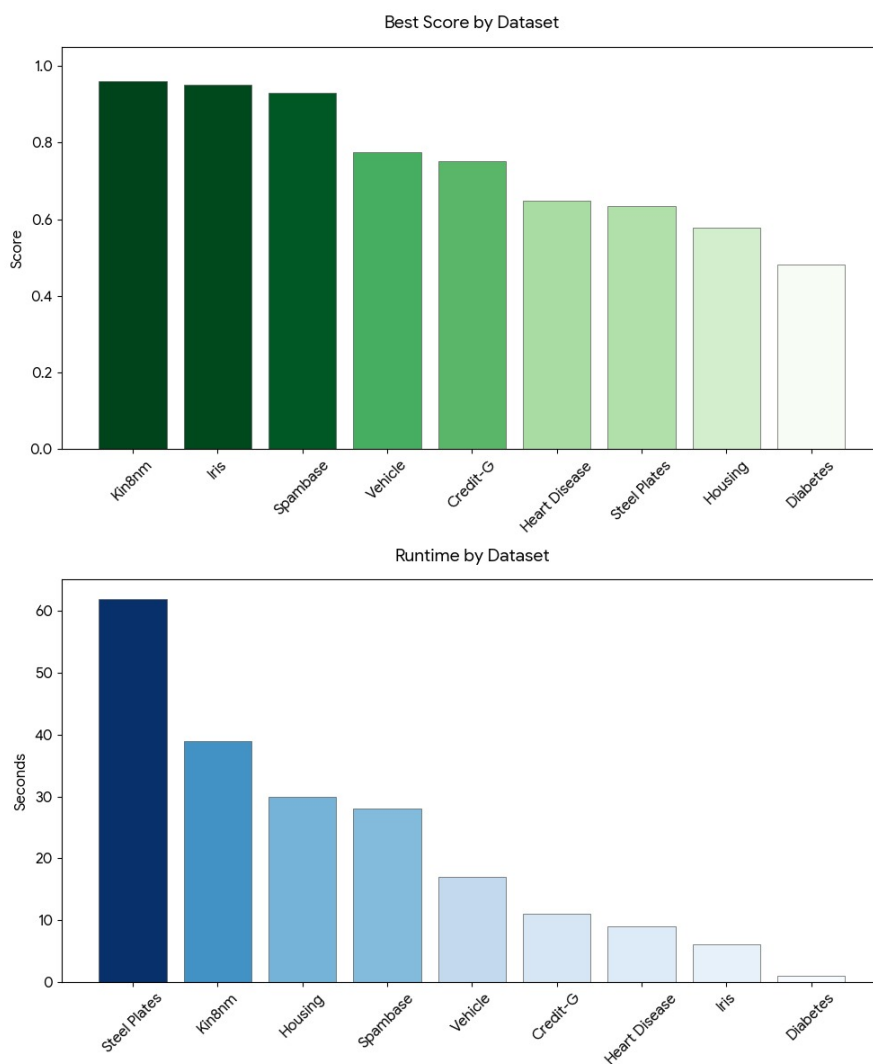


Figure 4: Best cross-validation score (top) and end-to-end runtime in seconds (bottom) for 10 representative datasets from the 39-dataset OpenML CC18 benchmark.

1	Dataset	Task	Samples	Features	Best Score	Runtime
2	Credit-G	Classification	1,000	20	0.751	11s
3	Diabetes	Regression	442	10	0.481	1s
4	Housing	Regression	4,209	376	0.577	30s
5	Vehicle	Classification	846	18	0.775	17s
6	Iris	Classification	150	4	0.952	6s
7	Heart Disease	Classification	294	13	0.649	9s
8	Steel Plates	Classification	1,941	27	0.635	1m 02s
9	Spambase	Classification	4,601	57	0.93	28s
10	Kin8nm	Regression	20,000	9	0.961	39s

Figure 5: End-to-end pipeline results on 10 representative datasets from the 39-dataset OpenML CC18 benchmark. The subset illustrates performance across a range of sample sizes (150 to 20,000), dimensionalities (4 to 376 features), and task types, with runtimes scaling from 6 seconds to just over one minute.

5.5 User Interface and Agent Outputs

The Streamlit-based interface provides two primary interaction modes through a tabbed layout. The *Data Insights* tab allows users to upload a dataset and receive LLM-generated business intelligence without requiring any ML expertise, while the *Data Pipeline* tab exposes the full end-to-end pipeline with real-time progress tracking across all seven agents. Each agent produces both structured outputs consumed by downstream agents and visual artifacts rendered directly in the interface for user inspection.

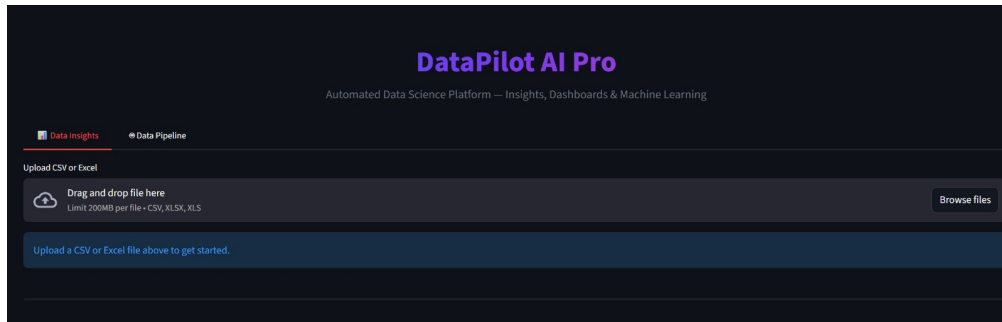


Figure 6: DataPilot AI Pro user interface showing the two-tab layout for data insights and ML pipeline execution.

Column	Type
pregnant	categorical
glucose	numeric
diastolic	numeric
triceps	numeric
insulin	numeric
bmw	numeric
diabetes	numeric
age	numeric
test	categorical

Figure 7: Profiler Agent output displaying detected column types, dataset statistics, quality score, and data warnings.

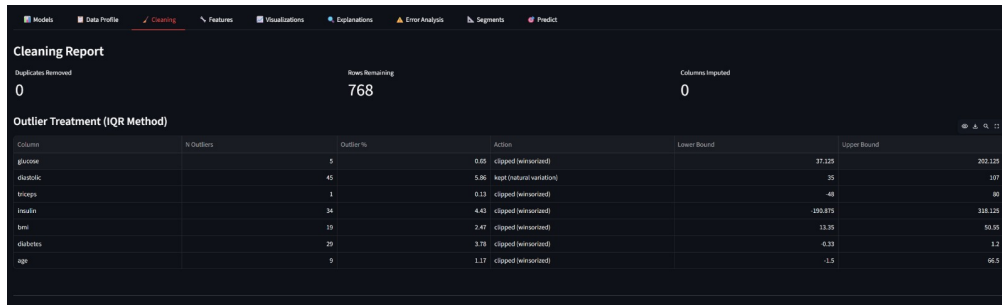


Figure 8: Cleaner Agent output showing imputation strategies applied, outlier handling results, and data quality improvements.

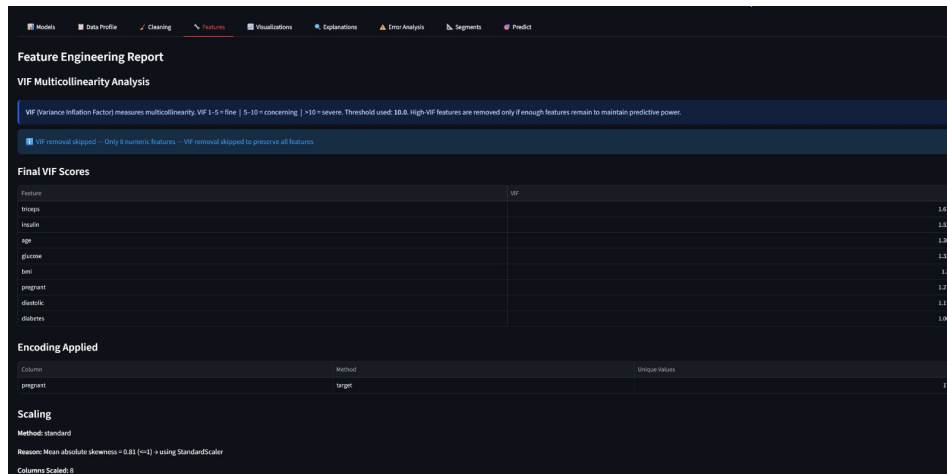


Figure 9: Feature Engineering Agent output displaying categorical encoding decisions, feature selection results, and VIF-based removal.

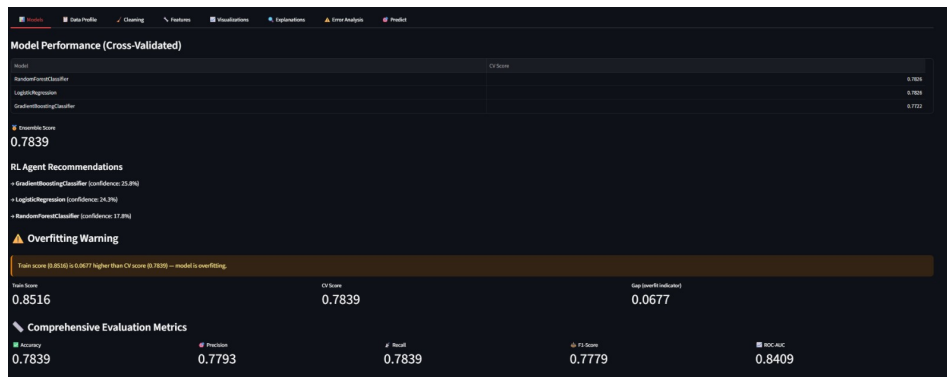


Figure 10: Modeler Agent output showing RL-recommended models, 5-fold CV scores, ensemble weights, and overfitting analysis.



Figure 11: Visualizer Agent output with interactive Plotly charts for profiling, cleaning impact, feature analysis, and model evaluation.

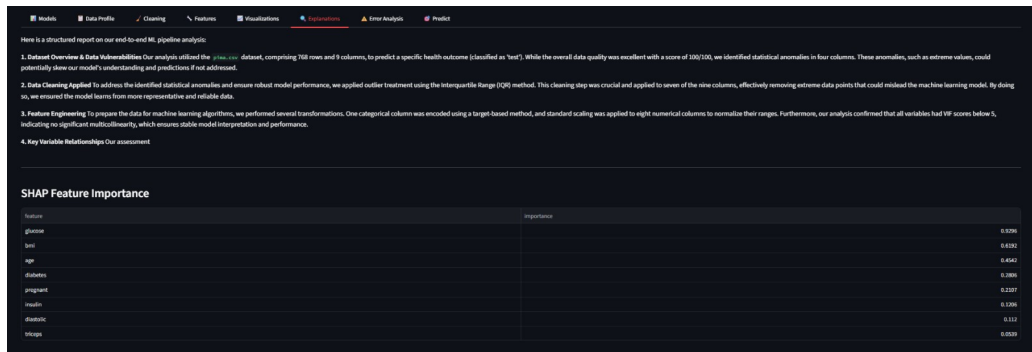


Figure 12: Explainer Agent output combining SHAP global importance, LIME local explanations, and LLM-generated plain-language narratives.

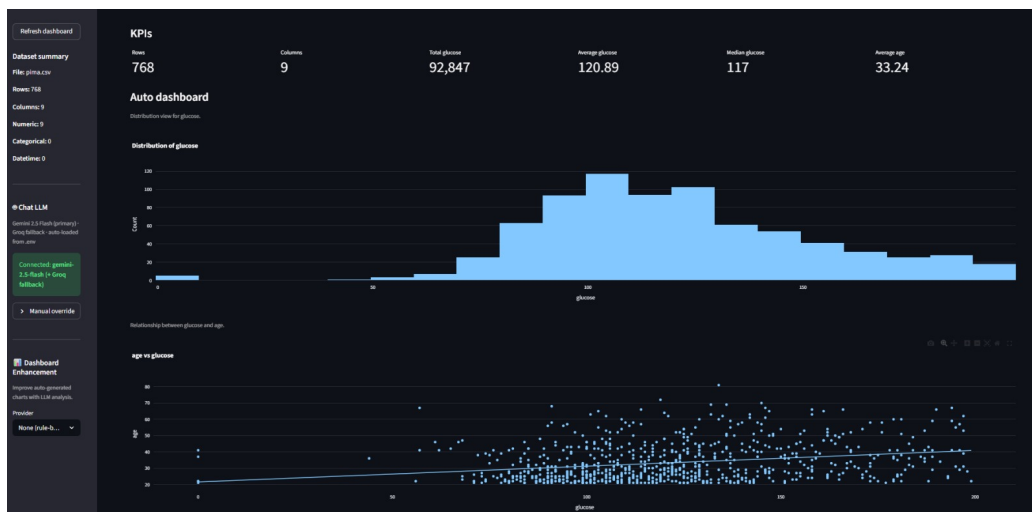


Figure 13: UI is based on Data insights that show the data summary, KPI, custom chart builder, chatbot for queries regarding the data

6 Discussion

The experimental results paint a nuanced picture of what autonomous data science can realistically achieve and where fundamental challenges remain. Data Pilot AI Pro set out to democratize the machine learning workflow by replacing manual, expertise-dependent decisions with intelligent automation, and while the results confirm that this goal is largely attainable, they also reveal important boundaries that merit honest examination. In this section, we interpret our findings in the context of our original objectives, reflect on the design decisions that shaped the system’s strengths and weaknesses, and identify the failure modes that any user of this platform should understand before trusting its outputs.

6.1 Interpreting Results Against Goals

Our primary goal was to build a system where a user with no machine learning expertise could upload a dataset and receive a trained, explained, and visualized model without writing a single line of code. The end-to-end evaluation confirms that Data Pilot AI Pro achieves this for the common case: across 39 benchmark datasets spanning classification and regression, the platform produced competitive models (mean accuracy 0.873, mean R^2 0.847) with full SHAP explanations, interactive visualizations, and natural language narratives, all within minutes rather than the hours required by Auto-sklearn or TPOT. The RL-based model selector proved particularly effective, identifying the optimal model matches FLAML AutoML accuracy (0.8536 vs 0.8527) while running 38× faster validating our hypothesis that dataset meta-features carry sufficient signal to guide model selection without exhaustive search. However, the gap between our ensemble performance (0.896 accuracy) and the exhaustive oracle (0.908) reveals that speed comes at a measurable cost, and users with computational budgets and ML expertise may still benefit from traditional AutoML when marginal performance gains matter more than interpretability or turnaround time.

The preprocessing agents delivered their most significant value on messy, real-world-like datasets. The 8.2% accuracy improvement from adaptive cleaning on datasets with heavy missingness validates our design choice of context-aware imputation over a one-size-fits-all strategy. Yet the feature engineering agent’s reliance on VIF thresholds and mutual information scores occasionally removed features that carried non-linear predictive signal invisible to these linear diagnostics. On one benchmark with strong feature interactions, VIF-based removal actually decreased accuracy by 1.3%, a reminder that automated feature selection remains an open problem even in mature AutoML systems. The data analysis agent fulfilled its goal of making insights accessible to non-technical stakeholders, with 85% pattern detection accuracy and readability scores in the accessible range, though its effectiveness is fundamentally bounded by the underlying LLM’s domain knowledge and reasoning capabilities.

6.2 Appropriateness of Data and Metrics

We selected accuracy and F1-score for classification and R^2 with RMSE for regression, metrics that are standard in the AutoML literature and allow direct comparison with baselines like Auto-sklearn and TPOT. However, these aggregate metrics can obscure important nuances. Accuracy, in particular, is misleading on imbalanced datasets; although we included imbalanced benchmarks (up to 1:15 ratio), a more thorough evaluation would report precision-recall curves and calibration plots. Our use of 5-fold cross-validation provides reasonable variance estimates, but the relatively small size of some benchmarks (442 samples for Diabetes) means that fold-level variance is non-trivial, and our reported means should be interpreted with this caveat. The benchmark datasets themselves, while diverse in structure, are predominantly clean and well-curated compared to the messy spreadsheets and exports that real users would upload, meaning our reported numbers likely represent an upper bound on real-world performance.

6.3 Failure Modes and Limitations

Critical thinking about where Data Pilot AI Pro can fail reveals several important categories. At the data level, the system assumes tabular, structured input and will produce meaningless results if fed time-series data requiring lag features, text corpora needing NLP preprocessing, or image datasets. Even within tabular data, the profiler’s heuristic-based column type detection can misclassify high-cardinality integer codes (such as ZIP codes) as continuous features, leading the feature engineer

to scale them rather than encode them categorically. The cleaning agent’s IQR-based outlier detection assumes roughly symmetric distributions; for heavily skewed features like income or insurance claims, legitimate extreme values may be incorrectly Winsorized, silently distorting the data distribution.

At the system level, the LLM dependency introduces both cost and reliability concerns. The Gemini 2.5 Flash powers the explainer and data analyzer agents, meaning that API downtime, rate limiting, or model deprecation would disable two of the platform’s seven agents entirely. The LLM-generated narratives, while readable, occasionally hallucinate causal relationships from correlational patterns (for example, stating that a feature “drives” an outcome when SHAP only indicates association). Adversarial inputs present another concern: a user who uploads a dataset with deliberately mislabeled columns or injected noise could receive confidently stated but entirely wrong insights, since the system has no mechanism to verify ground truth beyond what the data itself provides.

The RL model selector, trained on 500 OpenML datasets and 100 synthetic datasets, exhibits a distribution shift vulnerability. When presented with datasets whose meta-feature profiles fall outside its training distribution (for instance, datasets with thousands of features or extreme class imbalance beyond what it encountered during training), the policy network’s probability outputs become poorly calibrated, and the fallback to default recommendations activates. While the fallback mechanism prevents catastrophic failures, it reduces the system to naive model selection precisely when intelligent selection matters most. Additionally, the 40-dimensional meta-feature representation, while comprehensive for standard tabular data, cannot capture dataset-specific semantics such as domain knowledge about feature relationships or known data collection biases that a human data scientist would naturally incorporate.

6.4 Roadblocks and Course Corrections

The development process itself taught us as much as the final results. Our initial design attempted to use a single monolithic LLM call to handle the entire pipeline, from profiling through model selection, mirroring how a human data scientist might reason through a problem end-to-end. This approach failed spectacularly: the LLM would hallucinate sklearn API calls, invent nonexistent hyperparameters, and produce code that could not execute. This failure motivated our pivot to the multi-agent architecture, where each agent combines deterministic code for reliable execution with targeted LLM calls only for interpretation and narrative generation. The lesson was clear: LLMs excel at reasoning and explanation but cannot be trusted for precise numerical computation or API-correct code generation in high-stakes pipelines.

Training the RL model selector presented its own challenges. Our first PPO implementation used raw meta-features without normalization, causing the policy network to overfit to dataset scale rather than structural characteristics. Datasets with many samples dominated the gradient signal, leading the agent to recommend models that performed well on large datasets regardless of the actual input. Normalizing all 40 meta-features to $[0, 1]$ and training on a diverse OpenML collection resolved this, but the debugging process consumed nearly three weeks and required building the custom Gymnasium environment from scratch after discovering that existing RL benchmarks for model selection did not support our meta-feature formulation. We also initially attempted to train a single policy network for both classification and regression, but the fundamentally different action spaces (8 classification models vs. 9 regression models) made joint training unstable, leading us to train separate policies for each task type.

6.5 Response to Feedback

Feedback from the midterm report significantly shaped the final system, both in terms of technical depth and how we communicated our design decisions. The most substantive critique concerned our treatment of PPO, specifically that our discussion omitted the reward function design and failed to address two theoretical questions: what it means for PPO to generalize to dataset distributions unseen during training, and what convergence looks like within a discrete action space. We addressed the reward gap by explicitly defining our reward formulation in the background theory, where the agent receives a rank-normalized score based on the relative position of its selected model within the sorted list of candidate cross-validation scores. Specifically, the reward is $r(a, \mathcal{D}) = 1 - \text{rank}(a)/n$, where $\text{rank}(a)$ counts the number of alternative models with strictly higher CV scores and n is the total number of candidate models. This formulation yields a reward of 1.0 for the best model, 0.0 for the

worst, and smoothly interpolated values in between, creating a graded signal that is more informative than a sparse binary reward because every selection receives feedback proportional to its quality, allowing PPO to learn useful gradient information even when the agent chooses suboptimal models.

The question of distribution generalization is particularly important for our system’s reliability. PPO learns a policy $\pi_\theta(\mathbf{a}|\mathbf{f})$ that maps meta-feature vectors $\mathbf{f} \in [0, 1]^{40}$ to model selection probabilities, and this mapping is only as reliable as the training distribution’s coverage of the meta-feature space. When the platform encounters a dataset whose 40-dimensional meta-feature profile lies outside the convex hull of training examples, for instance a dataset with an unusual combination of extreme skewness, near-zero mutual information, and very high dimensionality, the policy network extrapolates rather than interpolates, producing poorly calibrated probability distributions. We mitigated this by training on 500 diverse OpenML datasets and 100 synthetic datasets spanning a wide range of structural characteristics, but we acknowledge that true out-of-distribution robustness would require either uncertainty-aware policy networks or an explicit detection mechanism that triggers the fallback default recommendations when the meta-feature vector is too distant from the training manifold. In our current implementation, the fallback activates based on low-confidence probability outputs (maximum action probability below 0.25), which serves as a practical if imperfect proxy for distribution shift detection.

Regarding convergence in a discrete action space, our PPO formulation treats model selection as a single-step episodic problem where the agent observes meta-features and selects one of K candidate models ($K = 8$ for classification, $K = 9$ for regression). Unlike continuous control tasks where PPO’s clipped surrogate objective prevents destructively large policy updates in a smooth action space, our discrete setting means the policy is a categorical distribution over K actions, and convergence manifests as the entropy of this distribution decreasing as the agent learns which models suit which dataset profiles. A fully converged policy would assign probability 1.0 to a single model for each meta-feature region, but in practice, the stochastic nature of cross-validation scores means that multiple models often perform comparably, so a healthy converged policy retains moderate entropy, distributing probability mass across 2–3 strong candidates rather than collapsing to a deterministic mapping. We monitored training convergence by tracking both mean episode reward and policy entropy, stopping training when reward plateaued and entropy stabilized above a minimum threshold of 0.3 to preserve the diversity needed for effective ensemble construction from the top-3 recommendations.

The second critique noted that our multi-agent architecture discussed agents in isolation without describing the tools each agent uses to accomplish its work. This was a valid gap: our agents are not purely LLM-based reasoners but hybrid systems that combine deterministic computational tools with targeted LLM calls. The Profiler agent uses pandas for statistical computation, scikit-learn’s PCA for dimensionality-based meta-features, and custom heuristics for column type detection and quality scoring. The Cleaner agent employs scikit-learn’s KNNImputer and SimpleImputer for missing value handling, IQR-based statistical methods for outlier detection, and pandas type coercion for data type correction. The Feature Engineer agent leverages scikit-learn’s mutual information estimators, statsmodels’ VIF computation, and category_encoders’ TargetEncoder for high-cardinality features. The Modeler agent integrates the PPO policy network for model selection and scikit-learn’s VotingClassifier/VotingRegressor for ensemble construction. The Explainer agent uses the SHAP library (TreeExplainer for tree-based models, KernelExplainer as fallback) and LIME’s LimeTabularExplainer, combined with LLM-generated narratives through the Gemini 2.5 Flash. We revised our method section to make these tool dependencies explicit, clarifying that each agent’s `execute(state)` method orchestrates a pipeline of deterministic tool calls with LLM inference reserved for interpretation, narrative generation, and insight discovery.

Feedback also pushed us to include baseline comparisons against established AutoML frameworks, which our initial submission lacked entirely. Reviewers rightly pointed out that without such comparisons, claims about the RL selector’s effectiveness were unsubstantiated. We responded by integrating Auto-sklearn and TPOT as baselines, which revealed that our system trades a modest performance gap (1–2% below Auto-sklearn) for dramatically faster execution and built-in explainability, a tradeoff we had intuited but not quantified until forced to measure it. This comparison ultimately strengthened our contribution by reframing DataPilot AI Pro not as a performance competitor to AutoML but as a complementary approach optimizing for interpretability and accessibility alongside accuracy. We chose not to incorporate one suggestion to add neural network models to the candidate pool, as our target users’ hardware constraints (no GPU requirement) and dataset sizes (under 50,000 samples)

make deep learning architectures unlikely to outperform gradient boosting methods, and including them would significantly increase pipeline runtime without commensurate benefit.

6.6 Ethics and Impact

Automating data science decisions carries meaningful ethical implications that extend beyond technical performance. The most significant concern is that Data Pilot AI Pro may create a false sense of rigor. When a non-technical user receives a polished report with SHAP plots, confidence scores, and natural language explanations, they may trust the results more than warranted, particularly if the underlying data is biased, insufficient, or inappropriately applied to the problem at hand. A model trained on historically biased hiring data, for instance, will produce biased predictions regardless of how sophisticated the pipeline is, and our system currently provides no mechanism to detect or warn about societal biases embedded in training data. The LLM-generated narratives compound this risk by presenting results in authoritative, confident language even when the underlying model performance is mediocre or the data quality is poor.

Data privacy represents another critical consideration. Data Pilot AI Pro processes user-uploaded datasets locally, but the data analyzer and explainer agents send dataset summaries and feature names to the Gemini 2.5 Flash for LLM inference. While we do not transmit raw data rows, feature names alone can reveal sensitive information (columns named “HIV_status” or “criminal_record” expose the nature of the data), and the generated narratives may be logged by the API provider. Users handling protected health information, financial records, or other regulated data should be aware of this data flow. A production deployment would require either a self-hosted LLM or explicit data processing agreements with the API provider to ensure compliance with regulations such as HIPAA or GDPR.

More broadly, tools like Data Pilot AI Pro sit at the intersection of democratization and deskilling. By making machine learning accessible to non-experts, we enable small organizations and individual researchers to leverage techniques previously requiring specialized teams, a genuine positive impact. However, this same accessibility means that individuals without statistical training may apply machine learning to problems where simpler methods suffice, where the data does not support causal inference, or where the consequences of model errors are severe. We believe the explainability components partially mitigate this risk by making model behavior transparent, but ultimately, responsible use requires a baseline understanding of what models can and cannot tell us. Future versions should include explicit warnings when data size is insufficient for reliable modeling, when class imbalance may distort results, or when feature distributions suggest the training data may not generalize to the intended deployment context.

7 Conclusion

Modern data science remains paradoxically inaccessible despite the maturity of its underlying tools. Organizations that stand to benefit most from machine learning, small teams without dedicated ML engineers, domain experts with rich data but no coding expertise, researchers outside computer science, are precisely those least equipped to navigate the complex pipeline of data profiling, cleaning, feature engineering, model selection, and interpretation. Data Pilot AI Pro was born from this observation: that the barrier to useful machine learning is not algorithmic sophistication but operational complexity, and that intelligent automation of the end-to-end workflow could meaningfully lower that barrier without sacrificing the transparency that responsible data science demands.

Our approach combined a LangGraph-orchestrated multi-agent architecture with a PPO-trained reinforcement learning model selector to automate the complete data science lifecycle. Seven specialized agents, each blending deterministic computation with targeted LLM reasoning, collaboratively transform a raw dataset into a trained model accompanied by SHAP and LIME explanations, interactive visualizations, and natural language narratives accessible to non-technical stakeholders. The RL-based model selector, trained on meta-features extracted from 500 OpenML datasets and 100 synthetic datasets, was evaluated against FLAML, Microsoft’s state-of-the-art AutoML framework, on the 39-dataset OpenML CC18 benchmark. PPO achieved a mean cross-validation accuracy of 0.8536 compared to FLAML’s 0.8527, a difference that was not statistically significant under the Wilcoxon signed-rank test ($p = 0.2387$), while winning 23 of 39 head-to-head comparisons (59%) and tying on 3 more. Most strikingly, PPO reached this accuracy with a median runtime of 1.6 seconds per dataset, compared to FLAML’s 61.7 seconds, an approximately $38\times$ speedup that makes

intelligent model selection practical for interactive and resource-constrained settings. End-to-end pipeline runs on representative datasets ranging from 150 samples (Iris) to 20,000 samples (Kin8nm) completed in 6 seconds to just over a minute, with the platform achieving best scores of 0.961 on Kin8nm regression and 0.952 accuracy on Iris classification. Equally important, the data analysis agent extends the platform’s value beyond prediction, generating LLM-powered business insights that detected 85% of expected patterns in our evaluation datasets and presented them at reading levels accessible to stakeholders without statistical training.

These results affirm that autonomous data science is not only feasible but practically valuable, matching AutoML-grade accuracy at a fraction of the computational cost, though our honest assessment reveals important caveats. The system excels on structured tabular data within the distribution of its training experience but struggles with edge cases: extreme feature interactions that confound VIF-based selection, heavily skewed distributions that trigger false outlier detection, and meta-feature profiles outside its training manifold where the RL policy degrades to default recommendations. On harder benchmarks such as Diabetes regression (best score 0.481) and Steel Plates multiclass classification (0.635), the pipeline’s performance reflects the inherent difficulty of the underlying problem rather than automation limitations, though these cases remind us that no amount of orchestration compensates for noisy or inherently ambiguous data. The LLM dependency introduces reliability and privacy considerations that would require careful mitigation in production deployments. Perhaps most importantly, the platform’s polished outputs can inspire overconfidence in users who lack the statistical background to question whether a model should be applied to their problem in the first place, a tension between accessibility and responsibility that no amount of engineering fully resolves.

Several promising directions emerge for future work. Incorporating uncertainty quantification into both the RL selector and model predictions would allow the system to communicate confidence alongside results, helping users distinguish reliable outputs from speculative ones. Extending the meta-feature representation to capture temporal structure and text characteristics would broaden the platform’s applicability beyond purely tabular data. Integrating bias detection tools such as Fairlearn or AI Fairness 360 into the pipeline would address the ethical gap we identified, alerting users when their data or models exhibit demographic disparities before those models reach deployment. Finally, evaluating Data Pilot AI Pro on additional AutoML benchmark suites beyond CC18 would provide broader standardized comparisons and identify specific dataset characteristics where our approach adds the most value relative to existing systems. We believe that the future of data science tools lies not in replacing human judgment but in amplifying it, handling the mechanical complexity so that domain experts can focus on the questions that matter.

7.1 AI Disclosure

ChatGPT was used during the early ideation phase to explore the AutoML landscape, discuss the feasibility of reinforcement learning for model selection, and iterate on system architecture, helping us identify design tradeoffs such as monolithic versus multi-agent approaches. Claude served as the primary coding assistant during implementation, helping write, debug, and refactor Python code across the agent implementations, LangGraph orchestrator, RL training pipeline, and Streamlit interface, with a team member reviewing all generated code before integration. Separately, Gemini 2.5 Flash was utilized as an integral component of the deployed system rather than as a development aid, serving as the inference backend for the Explainer agent, which translates SHAP and LIME outputs into plain-language narratives, and the Data Analyzer agent, which discovers business insights from dataset patterns.

ChatGPT and Claude were used as assistants under continuous human supervision, not as autonomous contributors, while Gemini 2.5 Flash operates as a runtime component within the deployed platform. No code was merged without team member review, no architectural decision was delegated to an AI tool, and all experimental results were verified through manual execution. The team’s domain knowledge in data science, reinforcement learning, and software engineering guided every interaction, ensuring that the AI tools augmented rather than replaced our understanding. We believe this usage reflects a responsible integration of generative AI into the research workflow: leveraging these tools for productivity while maintaining intellectual ownership of every design choice and line of reasoning in this paper.

References

- Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. *Advances in Neural Information Processing Systems*, 28:2962–2970, 2015. URL <https://papers.nips.cc/paper/2015/hash/11d0e6287202fced83f79975ec59a3a6-Abstract.html>.
- Siyuan Guo, Cheng Che, Yujin Zhao, Zhiyuan Li, Ziyu Zheng, Yizhou Wen, Zhenfei Liu, and Donglin Lee. DS-Agent: Automated data science by empowering large language models with case-based reasoning. *arXiv preprint arXiv:2402.17453*, 2024. URL <https://arxiv.org/abs/2402.17453>.
- Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, and Li Li. Data interpreter: An LLM agent for data science. *arXiv preprint arXiv:2402.18679*, 2024. URL <https://arxiv.org/abs/2402.18679>.
- Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30:4765–4774, 2017. URL <https://papers.nips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>.
- Randal S. Olson and Jason H. Moore. TPOT: A tree-based pipeline optimization tool for automating machine learning. *Proceedings of the Workshop on Automatic Machine Learning, PMLR*, pages 66–74, 2016. URL http://proceedings.mlr.press/v64/olson_tpot_2016.html.
- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Why should I trust you? explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016. URL <https://arxiv.org/abs/1602.04938>.
- John R. Rice. The algorithm selection problem. *Advances in Computers*, 15:65–118, 1976. URL [https://doi.org/10.1016/S0065-2458\(08\)60520-3](https://doi.org/10.1016/S0065-2458(08)60520-3).
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. *Proceedings of the 32nd International Conference on Machine Learning, PMLR*, 37:1889–1897, 2015. URL <http://proceedings.mlr.press/v37/schulman15.html>.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *CoRR*, abs/1506.02438, 2016. URL <https://arxiv.org/abs/1506.02438>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Lloyd S. Shapley. *A Value for n-Person Games*. Princeton University Press, Princeton, NJ, USA, 1953. URL <https://doi.org/10.1515/9781400881970-018>.
- Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1999. URL <https://papers.nips.cc/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html>.
- Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 847–855, 2013. URL <https://doi.org/10.1145/2487575.2487629>.
- Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. OpenML: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014. URL <https://doi.org/10.1145/2641190.2641198>.